

单位代码： 10293 密 级： 机密

南京邮电大学

专业学位硕士学位论文



论文题目： 基于 LangGraph 的 LLM 辅助逆向分析系统设计

学 号	<u>1025040920</u>
姓 名	<u>刘钰枫</u>
导 师	<u>王化群</u>
专业学位类别	<u>硕士</u>
类 型	<u>全日制</u>
专业（领域）	<u>网络空间安全</u>
论文提交日期	<u>2026.6.1</u>

摘要

随着大语言模型在代码理解、工具调用和多轮规划方面的能力不断增强，将其引入软件逆向分析已成为网络安全研究中的重要方向。传统逆向分析依赖专家在反汇编、反编译、函数命名、控制流恢复、协议识别和漏洞定位之间反复切换，知识密集、路径冗长且难以复现。LangGraph 以有状态图为核心组织节点、边、检查点和人工干预流程，为构建可追踪、可回放、可扩展的 LLM 辅助分析系统提供了较好的工程抽象。本文围绕“基于 LangGraph 框架设计的 LLM 辅助逆向分析系统”展开综述，首先梳理 LLM 智能体、检索增强生成、工具学习和多智能体协作的发展脉络；其次分析静态逆向、动态调试、二进制相似性、漏洞发现和恶意代码分析等任务对 LLM 系统提出的需求；然后归纳基于 LangGraph 的系统架构，包括任务图建模、工具层封装、证据记忆、人工确认、安全隔离和结果报告生成；最后讨论幻觉、上下文窗口、数据泄露、评测基准和人机协同等开放问题。综述认为，LLM 不能替代逆向工程师的证据判断，但可以通过结构化工作流程降低重复劳动、提高分析覆盖率，并使复杂逆向过程具备更强的可审计性。

关键词：大语言模型，LangGraph，逆向工程，二进制分析，智能体，检索增强生成

Abstract

Large language models have shown growing potential in code understanding, tool use, and multi-step reasoning, which makes them attractive for software reverse engineering and security analysis. Traditional reverse engineering requires analysts to move repeatedly among disassembly, decompilation, function renaming, control-flow recovery, protocol inference, and vulnerability triage. The process is knowledge-intensive, difficult to reproduce, and highly dependent on expert experience. LangGraph organizes LLM applications as stateful graphs with explicit nodes, edges, checkpoints, memory, and human-in-the-loop control, providing a practical abstraction for building traceable and extensible LLM-assisted reverse engineering systems. This survey reviews the design of such systems based on LangGraph. It first summarizes the foundations of LLM agents, retrieval-augmented generation, tool learning, and multi-agent collaboration. It then analyzes the requirements of static analysis, dynamic debugging, binary similarity, vulnerability discovery, and malware analysis. On this basis, the paper presents a reference architecture covering graph-based task orchestration, tool wrappers, evidence memory, human confirmation, sandboxing, and report generation. Finally, it discusses open challenges including hallucination, context limits, data leakage, benchmark construction, and human-AI collaboration. The survey argues that LLMs should not replace expert judgment in reverse engineering, but they can reduce repetitive work, improve analysis coverage, and make complex investigations more auditable when embedded in structured workflows.

Key Words: Large Language Model, LangGraph, Reverse Engineering, Binary Analysis, Agent, Retrieval-Augmented Generation

目 录

摘要	I
Abstract	II
表格索引	IV
第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.3 研究内容与论文结构	2
1.4 本章小结	2
第二章 相关技术基础	3
2.1 大语言模型与智能体范式	3
2.2 LangGraph 框架	3
2.3 逆向工程与二进制分析基础	3
2.4 检索增强生成与知识库	4
2.5 本章小结	4
第三章 LLM 辅助逆向分析研究综述	5
3.1 任务类型划分	5
3.2 代码理解与反编译增强	5
3.3 漏洞发现与安全审计	5
3.4 恶意代码分析与行为归纳	6
3.5 现有研究的不足	6
3.6 本章小结	6
第四章 基于 LangGraph 的系统架构设计	7
4.1 总体设计目标	7
4.2 分层架构	7
4.3 状态图建模	7
4.4 多智能体协作机制	8
4.5 证据链与报告生成	8
4.6 本章小结	9
第五章 关键技术、评测与挑战	10
5.1 工具封装与上下文压缩	10
5.2 提示工程与输出约束	10
5.3 评测指标设计	10
5.4 安全风险与治理	11
5.5 开放挑战	11
5.6 本章小结	11
第六章 总结与展望	12
6.1 本文工作总结	12
6.2 进一步研究方向	12
参考文献	13
附录 A 程序清单	14

表格索引

表 4.1 系统主要模块及职责 7

表 5.1 LLM 辅助逆向系统评测维度 11

第一章 绪论

1.1 研究背景与意义

软件逆向工程是网络空间安全、漏洞挖掘、恶意代码分析和数字取证中的基础技术。分析人员通常需要在缺少源代码、符号信息和设计文档的条件下，从二进制程序、运行轨迹、网络行为和系统调用中恢复程序语义。传统工具如 IDA Pro、Ghidra、Binary Ninja、radare2、angr 等能够提供反汇编、反编译、控制流图、符号执行和脚本化自动化能力，但它们更擅长呈现事实和执行确定性分析，较少直接承担跨函数语义归纳、假设生成、证据整理和报告撰写等认知任务。

大语言模型（Large Language Model, LLM）在自然语言理解、代码补全、解释生成和上下文推理方面取得显著进展。GPT-4 技术报告显示，通用大模型在复杂问题求解与代码任务上已经具备较强的迁移能力^[1]。ReAct 将推理与行动交替组织，使模型能够在思考过程中调用外部工具并根据观察结果修正计划^[2]；Toolformer 证明语言模型可以学习何时使用外部工具^[3]；检索增强生成（Retrieval-Augmented Generation, RAG）则通过外部知识库缓解模型知识陈旧和事实错误问题^[4]。这些研究共同推动了“LLM 智能体”范式的发展。

在逆向工程场景中，LLM 的价值并不只是“解释一段伪代码”。更重要的是，它可以把工具输出、领域知识和分析目标组织成可迭代的工作流：先识别入口函数和关键 API，再根据字符串、导入表、控制流和运行日志提出假设，随后调用反编译器、调试器、沙箱或检索系统验证假设，并将证据转化为结构化结论。LangGraph 将这种过程表示为有状态图，使节点能够对应具体分析动作，边能够表达条件转移，检查点能够保存中间证据，人工节点能够在高风险判断处接入专家确认。相比单轮问答或简单链式提示，图式工作流更适合逆向分析这种长流程、强证据、可回溯的任务。

1.2 国内外研究现状

LLM 与逆向分析结合的研究可以分为三类。第一类关注模型对代码和二进制表示的理解能力，例如利用反编译伪代码、汇编指令或中间表示进行函数命名、类型恢复和语义摘要。LLM4Decompile 将大模型用于二进制代码反编译，探索从汇编或低级表示恢复高级语言表达的能力^[5]。第二类关注安全任务，如漏洞检测、补丁分析和恶意行为解释。相关研究尝试将函数语义、调用图和静态特征输入模型，辅助定位潜在缺陷或生成漏洞描述。第三类关注智能体系统，即让模型围绕一个逆向目标调用工具、维护状态并生成可验证结论。ReCopilot 等工作体现了“逆向分析助手”的趋势：模型不再只是回答问题，而是在工具链中承担规划、导

航和解释角色^[6]。

另一方面，逆向分析本身已有成熟的自动化研究基础。`angr` 提供基于中间表示的程序分析平台，支持符号执行、控制流恢复和漏洞挖掘^[7]；`Ghidra` 作为开源逆向工程框架，提供反编译器和丰富的脚本接口；`IDA Pro` 仍是工业界广泛使用的交互式反汇编平台。这些工具的共同特点是结果可验证、执行路径明确，但使用门槛较高。LLM 系统若要可靠地进入这一领域，必须尊重既有工具产生的证据，而不是以自然语言生成替代严谨分析。

`LangGraph` 的出现为上述问题提供了工程组织方式。其官方文档将 `LangGraph` 定位为面向长期运行、有状态智能体的编排框架，强调持久执行、人工介入、记忆和状态管理等能力^[8]。这与逆向工程中的“多步骤、长上下文、需复核”特征高度契合。因此，围绕 `LangGraph` 设计 LLM 辅助逆向分析系统具有现实意义。

1.3 研究内容与论文结构

本文是一篇面向系统设计的综述论文，核心问题是：如何利用 `LangGraph` 将 LLM 的语言推理能力、逆向工具的确定性分析能力和人工专家的判断能力组合为一个可审计的辅助系统。本文不把 LLM 视为独立分析者，而将其定位为工作流中的规划器、解释器、检索器和报告生成器。

全文结构如下。第二章介绍 LLM 智能体、`LangGraph` 和逆向工程基础。第三章梳理 LLM 辅助逆向分析的典型任务与技术路线。第四章提出基于 `LangGraph` 的系统架构设计。第五章讨论系统实现中的关键技术、评测方法和安全约束。第六章总结全文并展望未来研究方向。

1.4 本章小结

本章说明了 LLM 辅助逆向分析的研究背景、技术动因和工程挑战。逆向工程需要严谨证据链，LLM 则擅长跨源信息整合和自然语言表达。`LangGraph` 以图式状态机组织智能体流程，为二者结合提供了可落地的系统框架。

第二章 相关技术基础

2.1 大语言模型与智能体范式

大语言模型通过在大规模文本和代码语料上预训练，学习语言、代码和知识之间的统计关联。与早期面向单一任务的模型不同，现代 LLM 能够在提示词引导下完成摘要、问答、代码解释、计划生成和格式化输出等多种任务。Chain-of-Thought 提示证明，显式中间推理步骤可以提升模型在复杂任务中的表现^[9]。在安全分析中，这种能力可用于把“程序可能做了什么”拆解为若干可验证子问题。

智能体范式进一步将 LLM 放入“观察-思考-行动-反馈”的闭环中。ReAct 通过交替生成推理和动作，使模型能够调用搜索、计算或程序分析工具，并根据返回结果继续推理^[2]。Reflexion 引入语言形式的自我反馈，让智能体从失败轨迹中总结经验^[10]。AutoGen 则强调多智能体会话，将不同角色模型组织为协作系统^[11]。这些思想为逆向分析系统中的“规划智能体、工具智能体、审计智能体、报告智能体”分工提供了理论基础。

2.2 LangGraph 框架

LangGraph 是 LangChain 生态中面向有状态智能体和工作流的框架。其核心抽象包括状态、节点、边、条件转移、检查点和人工介入。状态用于保存当前任务、证据、工具输出、模型结论和错误信息；节点表示一次可执行操作，例如反编译函数、检索知识、调用 LLM 生成假设或要求人工确认；边定义节点之间的控制流；检查点则允许长任务在中断后恢复，并支持对关键分析过程进行回放^[8]。

在逆向分析中，LangGraph 的优势主要体现在四个方面。第一，分析流程显式化。系统可以把“样本导入、快速分诊、函数聚类、关键路径分析、漏洞假设验证、报告生成”表示为图，而非隐藏在一段长提示词中。第二，中间证据可保存。每个节点产生的反编译结果、字符串列表、调用图摘要和运行日志都可以进入状态或外部证据库。第三，错误处理更自然。若某个工具失败或模型结论置信度不足，条件边可以将任务转向重试、降级分析或人工审核。第四，便于组合不同角色。不同节点可以使用不同模型、不同提示词和不同权限，从而降低单一模型误判带来的风险。

2.3 逆向工程与二进制分析基础

逆向工程通常包括静态分析和动态分析。静态分析在不运行程序的情况下检查文件格式、导入导出表、字符串、反汇编代码、控制流图和数据流关系；动态分析通过调试、沙箱执行、

系统调用监控和网络流量捕获观察程序行为。两类方法互补：静态分析覆盖面广但可能受混淆和壳保护影响，动态分析证据直接但受路径覆盖和环境触发条件限制。

二进制分析工具链通常包含反汇编器、反编译器、调试器、符号执行引擎和脚本接口。IDA Pro 和 Ghidra 代表交互式逆向平台，angr 则代表自动化程序分析框架。对于 LLM 系统而言，这些工具不只是“外部插件”，而是事实来源。模型生成的任何结论都应能追溯到工具输出，例如函数地址、交叉引用、字符串常量、API 调用、污点传播路径或动态执行日志。

2.4 检索增强生成与知识库

逆向分析常依赖大量外部知识，包括 Windows API、Linux 系统调用、加密算法特征、编译器模式、漏洞类型、恶意软件家族行为和历史样本报告。仅依赖模型参数记忆容易产生过时或虚构结论。RAG 通过检索外部文档并将相关片段放入上下文，使模型回答更接近可验证知识^[4]。

面向逆向工程的知识库可以分为三层。第一层是通用安全知识，如 CWE、ATT&CK、系统 API 文档和常见加密算法特征。第二层是项目内知识，如已分析样本的函数摘要、变量命名和协议字段。第三层是运行时证据，如当前样本的反编译结果、调试日志和沙箱报告。LangGraph 可以在不同节点中按需检索这些知识，并把检索来源写入状态，形成可审计证据链。

2.5 本章小结

本章介绍了 LLM 智能体、LangGraph、逆向工程和 RAG 的基础概念。LLM 提供推理与表达能力，逆向工具提供可验证证据，LangGraph 则提供状态化编排机制。三者结合构成 LLM 辅助逆向分析系统的技术基础。

第三章 LLM 辅助逆向分析研究综述

3.1 任务类型划分

LLM 辅助逆向分析可按照任务粒度分为函数级、模块级和样本级三类。函数级任务关注单个函数的语义恢复，包括函数命名、参数含义推断、局部变量重命名、关键分支解释和伪代码摘要。模块级任务关注若干函数之间的协作关系，例如配置解析模块、加密通信模块、持久化模块和漏洞触发路径。样本级任务则面向完整程序或恶意代码样本，要求给出功能概述、行为链、风险评级和处置建议。

从分析目标看，常见任务包括：一是语义摘要，即把汇编或伪代码转化为自然语言说明；二是结构恢复，即识别数据结构、协议字段和状态机；三是漏洞分析，即结合数据流、边界检查和危险 API 判断缺陷；四是恶意行为解释，即识别持久化、反调试、命令控制、信息窃取和横向移动等行为；五是报告生成，即将证据整理为面向研究人员或应急响应人员的文档。

3.2 代码理解与反编译增强

反编译器能够把二进制代码恢复为接近高级语言的伪代码，但变量名、类型、结构体边界和业务语义往往缺失。LLM 可以在伪代码基础上补充语义解释，帮助分析人员快速理解函数意图。例如，对于包含循环、查表和异或操作的函数，模型可以结合常见算法模式判断其可能是校验、编码或轻量加密逻辑。LLM4Decompile 等研究进一步探索了直接面向低级代码的反编译能力，说明模型可以学习汇编模式与高级语言结构之间的映射^[5]。

但该方向也存在明显限制。模型可能把相似代码模式误判为特定算法，或在缺少调用上下文时生成过度自信的解释。因此，可靠系统应采用“候选解释—证据验证”模式：模型先提出函数名和语义假设，随后工具节点检查字符串、常量、调用者、被调用者和动态行为，最后再决定是否接受命名。

3.3 漏洞发现与安全审计

漏洞分析要求系统从输入源、传播路径、边界检查和危险操作之间建立因果关系。传统静态分析工具擅长构造控制流图和数据流图，但误报较多，解释成本高。LLM 可以帮助解释告警、归纳漏洞模式，并生成面向人工复核的审计报告。例如，当工具发现外部输入流向 strcpy、memcpy 或格式化字符串函数时，LLM 可以结合上下文判断长度约束是否充分，并提出后续验证步骤。

此类任务的核心不是让模型直接裁决“是否存在漏洞”，而是让模型降低人工审计的组织

成本。系统应记录每个结论所依赖的地址、基本块、调用链和变量关系，避免把自然语言推测误写为事实。对于高风险结论，应通过 LangGraph 的人工确认节点进入专家复核。

3.4 恶意代码分析与行为归纳

恶意代码分析强调从多源证据中恢复行为链。静态证据包括导入表、字符串、资源段、配置数据、加密常量和反调试逻辑；动态证据包括文件操作、注册表修改、进程注入、网络连接和命令控制协议。LLM 能够将这些分散证据整合成行为描述，并映射到 ATT&CK 技术项或处置建议。

与普通程序理解相比，恶意代码分析更加依赖对抗性假设。样本可能使用壳、混淆、环境检测、延迟执行和动态 API 解析。LLM 在这里适合作为分析调度器：当静态节点发现可疑字符串时，调度动态沙箱；当动态节点发现网络行为时，回溯相关配置解析函数；当模型解释无法覆盖证据时，触发更多工具分析。这种闭环能力正是 LangGraph 适合承载的部分。

3.5 现有研究的不足

现有 LLM 逆向研究仍面临若干不足。第一，评测数据集不统一，不同工作常使用自建样本或少量案例，难以比较。第二，许多实验偏重模型输出质量，而缺少对工具调用、证据追踪和人工复核成本的评估。第三，模型容易受到反编译器输出质量影响，面对混淆、内联汇编和异常控制流时鲁棒性不足。第四，安全边界尚不清晰，自动执行未知样本、上传敏感二进制或泄露专有代码都可能带来风险。

3.6 本章小结

本章从任务类型、代码理解、漏洞发现和恶意代码分析四个方面梳理了 LLM 辅助逆向分析研究。总体来看，LLM 的优势在于语义归纳和工作流调度，短板在于事实可靠性和底层执行能力。因此，系统设计必须坚持工具证据优先和人工可复核原则。

第四章 基于 LangGraph 的系统架构设计

4.1 总体设计目标

基于 LangGraph 的 LLM 辅助逆向分析系统应满足五个目标。第一，可追踪：每个结论都能追溯到工具输出、模型提示和人工确认记录。第二，可恢复：长时间分析任务可通过检查点中断和继续。第三，可扩展：系统能够接入不同反编译器、调试器、沙箱、知识库和模型。第四，可控：高风险操作必须经过权限控制和人工确认。第五，可评测：系统输出不仅包含自然语言报告，还包含结构化指标，便于比较不同策略。

4.2 分层架构

系统可划分为输入层、工具层、知识层、智能体层、编排层和输出层。输入层负责接收二进制样本、符号文件、运行日志和用户目标。工具层封装反汇编、反编译、字符串提取、文件格式识别、调试执行、沙箱分析和符号执行等能力。知识层保存外部安全知识和项目内分析记忆。智能体层包含规划智能体、函数语义智能体、漏洞审计智能体、恶意行为智能体和报告智能体。编排层由 LangGraph 实现，负责状态更新、节点调度、条件分支和检查点。输出层生成函数注释、结构化证据表和综述式分析报告。

表 4.1 系统主要模块及职责

模块	职责
输入层	接收样本、哈希、用户问题、已有日志和分析范围约束
工具层	调用反编译、反汇编、字符串提取、沙箱、调试和符号执行工具
知识层	管理 API 文档、漏洞模式、历史样本、函数摘要和检索索引
智能体层	完成任务规划、语义解释、证据核验、风险判断和报告撰写
LangGraph 编排层	维护状态、执行节点、选择条件边、保存检查点并处理人工介入
输出层	生成函数命名建议、证据链、漏洞假设、行为摘要和最终报告

4.3 状态图建模

LangGraph 中的状态可以设计为一个结构化对象，包含样本元数据、任务目标、待分析函数队列、已确认事实、候选假设、工具调用记录、错误记录和最终报告草稿。节点则对应具体动

作。例如, `triage_sample` 节点完成文件类型、哈希、导入表和字符串提取; `select_functions` 节点根据入口点、交叉引用和可疑 API 选择优先函数; `decompile_function` 节点调用反编译器; `summarize_function` 节点生成函数摘要; `verify_hypothesis` 节点检查证据是否支持模型判断; `human_review` 节点等待人工确认。

算法 1 基于 LangGraph 的逆向分析流程

输入: 二进制样本 B , 分析目标 Q

```

1: 初始化状态  $S \leftarrow \{B, Q, \emptyset\}$ 
2: 执行样本分诊节点, 提取文件元数据、字符串和导入表
3: 根据入口点、可疑字符串和调用图选择候选函数
4: while 存在未分析候选函数 do
5:   调用反编译节点获得伪代码和控制流摘要
6:   调用 LLM 节点生成函数语义假设和命名建议
7:   调用验证节点检查地址、调用关系、常量和外部文档证据
8:   if 证据不足或风险较高 then
9:     转入人工复核节点或动态分析节点
10:  else
11:   将确认事实写入分析记忆
12:  end if
13: end while
14: 生成结构化证据表和自然语言分析报告
```

输出: 可审计逆向分析报告 R

4.4 多智能体协作机制

单一智能体容易在复杂任务中混淆职责。更稳妥的设计是将系统拆分为多个角色。规划智能体负责任务分解和优先级排序; 工具智能体只负责调用工具并返回原始结果; 语义智能体解释伪代码和汇编; 审计智能体检查结论是否被证据支持; 报告智能体负责最终表达。角色拆分可降低提示词复杂度, 也便于为不同节点配置不同模型和权限。

多智能体协作并不意味着所有节点都必须由 LLM 驱动。相反, 确定性工具应承担能够确定完成的工作, 例如哈希计算、字符串提取、反编译、图遍历和规则匹配。LLM 应承担开放性较强的任务, 如选择下一步分析方向、解释复杂逻辑、合并多源证据和生成报告。LangGraph 的条件边可以在工具结论明确时绕过模型节点, 从而节约成本并降低误差。

4.5 证据链与报告生成

逆向报告的可信度来自证据链。系统应把“结论-证据-来源-置信度-复核状态”作为基本输出单元。例如, “样本具有持久化行为”这一结论应关联到具体注册表路径、调用函数地址、动态日志时间戳和相关截图或命令行记录。报告智能体只允许基于已确认事实生成正文, 对未确认内容应标记为假设或待验证项。

4.6 本章小结

本章提出了基于 LangGraph 的 LLM 辅助逆向分析系统架构。该架构以状态图为核心，将逆向工具、知识库、多智能体和人工复核整合到统一流程中，使分析过程更可控、更可恢复，也更便于审计。

第五章 关键技术、评测与挑战

5.1 工具封装与上下文压缩

逆向工具输出通常体量庞大，不能直接全部放入模型上下文。系统需要对工具输出进行分层压缩。第一层保留原始证据，如完整伪代码、汇编片段和日志文件；第二层生成结构化摘要，如函数签名、调用者、被调用者、字符串、常量和危险 API；第三层生成面向 LLM 的上下文片段，仅包含当前问题相关信息。这样既能控制上下文长度，又能在需要时回溯原始证据。

工具封装应采用统一接口，记录命令参数、执行时间、返回码、标准输出、错误输出和结果文件路径。对于反编译器、调试器和沙箱这类高风险工具，应设置超时、资源限制和隔离环境。模型节点不应直接拼接任意命令，而应从白名单工具和参数模板中选择，以减少提示注入和越权执行风险。

5.2 提示工程与输出约束

逆向分析提示词应强调证据优先、边界清晰和不确定性表达。一个可靠提示通常包含任务目标、可用证据、禁止臆测规则、输出格式和置信度要求。对于函数摘要，可以要求模型输出“功能概述、关键输入、关键输出、外部调用、可疑行为、证据地址和待验证问题”。对于漏洞审计，可以要求模型区分“已确认缺陷、可疑风险、误报可能和下一步验证”。

输出格式宜采用 JSON、表格或固定小节，以便后续节点读取。若模型只生成自然语言段落，系统很难自动判断哪些结论已经确认、哪些需要复核。因此，LangGraph 状态中的事实、假设和问题应分开存储。报告生成阶段再将这些结构化内容转化为论文式或应急报告式文本。

5.3 评测指标设计

LLM 辅助逆向系统的评测不能只看回答是否流畅，而应覆盖准确性、效率、可审计性和安全性。准确性指标包括函数命名正确率、行为分类准确率、漏洞定位准确率和误报率。效率指标包括完成分析所需时间、工具调用次数、人工介入次数和 token 成本。可审计性指标包括结论证据覆盖率、可复现工具调用比例和报告中未证实判断比例。安全性指标包括沙箱逃逸防护、敏感数据外发控制和危险操作拦截率。

表 5.1 LLM 辅助逆向系统评测维度

维度	示例指标
准确性	函数命名正确率、漏洞定位准确率、行为分类准确率、误报率
效率	分析耗时、工具调用次数、人工复核次数、模型调用成本
可审计性	证据覆盖率、可复现调用比例、未证实结论比例、检查点完整性
鲁棒性	混淆样本表现、壳保护样本表现、反调试场景表现、长上下文稳定性
安全性	沙箱隔离、权限控制、敏感数据保护、提示注入抵抗能力

5.4 安全风险与治理

将 LLM 接入逆向工具链会引入新的安全风险。首先，样本本身可能是恶意程序，自动动态分析必须在隔离环境中进行。其次，二进制样本可能包含商业机密或未公开漏洞信息，上传到外部模型服务前需要进行脱敏和权限审查。再次，工具输出中可能包含攻击指令或诱导文本，模型若不加区分地执行可能受到提示注入影响。最后，自动化系统可能降低攻击门槛，因此应限制危险功能，例如自动生成可利用漏洞的完整利用代码。

治理措施包括最小权限原则、工具白名单、人工确认、日志审计、模型输出过滤和数据分级。对于教学和科研场景，系统应更多强调解释、复核和报告，而不是自动攻击能力。对于企业安全场景，系统还应纳入合规流程，明确数据存储、模型调用和结果共享边界。

5.5 开放挑战

当前仍有若干开放挑战。第一，模型幻觉尚未彻底解决，尤其是在反编译结果不完整时，模型容易生成看似合理但缺乏证据的解释。第二，长上下文分析成本高，跨上百个函数的全局推理仍困难。第三，混淆、虚拟化保护和动态代码生成会显著降低静态证据质量。第四，公开评测基准不足，难以系统比较不同工作流。第五，人机协作界面仍待探索，如何让分析人员快速理解模型依据并纠正错误，是工程落地的关键。

5.6 本章小结

本章讨论了工具封装、上下文压缩、提示工程、评测指标和安全治理。LLM 辅助逆向系统要真正可用，必须把模型能力嵌入受控流程，并以证据链、评测指标和安全边界约束其输出。

第六章 总结与展望

6.1 本文工作总结

本文围绕“基于 LangGraph 框架设计的 LLM 辅助逆向分析系统”进行了综述。首先，论文分析了逆向工程中知识密集、流程冗长和结果难复现等问题，指出 LLM 在语义归纳、任务规划和报告生成方面具有辅助价值。其次，论文梳理了 LLM 智能体、ReAct、RAG、工具学习、多智能体协作和 LangGraph 状态图等相关技术，并说明它们与逆向分析任务之间的对应关系。再次，论文从函数理解、反编译增强、漏洞审计和恶意代码行为归纳等角度总结了 LLM 辅助逆向分析的研究现状。最后，论文提出了一个基于 LangGraph 的分层系统架构，讨论了工具封装、证据记忆、人工复核、评测指标和安全治理等关键问题。

总体来看，LLM 辅助逆向分析的核心不是让模型独立替代专家，而是让模型在明确边界内承担规划、解释、检索和表达任务。LangGraph 提供的状态化编排、条件转移、检查点和人工介入机制，使复杂逆向过程能够从“临时对话”转变为“可审计工作流”。这种转变有助于降低重复劳动，提高分析覆盖率，并沉淀可复用的项目知识。

6.2 进一步研究方向

未来研究可以从五个方向展开。第一，构建面向逆向工程的标准化评测基准，覆盖函数命名、结构恢复、漏洞定位和恶意行为解释等任务。第二，探索更适合二进制代码的表示方式，将汇编、伪代码、控制流图、数据流图和动态轨迹共同输入模型。第三，发展面向证据链的 RAG 方法，使模型回答能够稳定引用地址、日志和工具输出。第四，完善人机协作界面，让分析人员能够快速查看模型依据、接受或驳回假设，并将反馈写回系统记忆。第五，加强安全治理，尤其是样本隔离、敏感数据保护、提示注入防护和自动化攻击能力限制。

随着模型能力、逆向工具接口和智能体框架的持续发展，LLM 辅助逆向分析有望从“问答式助手”演进为“可验证的分析协同系统”。在这一过程中，LangGraph 的价值在于把不稳定的模型推理放入稳定的工程流程之中，使每一步分析都有状态、有证据、有边界，也有人工专家可以介入的位置。

参考文献

- [1] OpenAI. GPT-4 Technical Report[J]. arXiv preprint arXiv:2303.08774, 2023.
- [2] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]. International Conference on Learning Representations. [S.l.: s.n.], 2023.
- [3] Schick T, Dwivedi-Yu J, Dessì R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools [J]. arXiv preprint arXiv:2302.04761, 2023.
- [4] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]. Advances in Neural Information Processing Systems: volume 33. [S.l.: s.n.], 2020: 9459–9474.
- [5] Tan H, Luo Q, Li J, et al. LLM4Decompile: Decompile Binary Code with Large Language Models[J]. arXiv preprint arXiv:2403.05286, 2024.
- [6] Chen G, Sun H, Liu D, et al. ReCopilot: Reverse Engineering Copilot in Binary Analysis[J]. arXiv preprint arXiv:2505.16366, 2025.
- [7] Shoshitaishvili Y, Wang R, Salls C, et al. SOK: (State of) The Art of War: Offensive Techniques in Binary Analysis[C]. 2016 IEEE Symposium on Security and Privacy. 2016: 138–157.
- [8] LangChain. LangGraph Documentation: Build Stateful Agents[M]. [S.l.: s.n.], 2026.
- [9] Wei J, Wang X, Schuurmans D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models[C]. Advances in Neural Information Processing Systems: volume 35. [S.l.: s.n.], 2022: 24824–24837.
- [10] Shinn N, Cassano F, Gopinath A, et al. Reflexion: Language Agents with Verbal Reinforcement Learning[C]. Advances in Neural Information Processing Systems: volume 36. [S.l.: s.n.], 2023.
- [11] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation[J]. arXiv preprint arXiv:2308.08155, 2023.

附录 A 程序清单

第三章 QAM 系统 CMA 算法程序 (QAM programme 文件夹)

mainQAM_MSE_CMAvsCADAMA: QAM 系统, 画 MSE 曲线主函数

mainQAM_SER_CMAvsCADAMA: QAM 系统, 画 SER 曲线主函数

mainDQAM_SER_PhaseRotate: 差分 QAM 系统, 相位旋转下, 画 SER 曲线主函数

mainDQAM_SER_Doppler: 差分 QAM 系统, 多普勒频移下, 画 SER 曲线主函数

mainDQAM_SER_SNR: 差分 QAM 系统, 不同 SNR 下, 画 SER 曲线主函数

第四章 QPSK 系统 CMA 算法程序 (PSK programme 文件夹)

mainDPSK_SER_PhaseRotate: 差分 PSK 系统, 相位旋转下, 画 SER 曲线主函数

mainDPSK_SER_Doppler: 差分 PSK 系统, 多普勒频移下, 画 SER 曲线主函数

mainDPSK_SER_SNR: 差分 PSK 系统, 不同 SNR 下, 画 SER 曲线主函数

FunQPSK: QPSK 编码子函数

FunDeQPSK_b: QPSK 解码子函数 (输出为比特)

第五章 CMA 频域均衡系统程序 (FDE programme 文件夹)

main_CMAFDE: CMA 频域均衡主函数

FunCAZACSeGen: 产生 CAZAC 序列子函数

Fun8PSK: 8PSK 调制子函数

FunDe8PSK_s: 8PSK 解调输出符号子函数

FunQPSK1: QPSK 调制幅度为 1 子函数

FunDeQPSK1_s: QPSK 解调输出符号幅度为 1 子函数

FunSDCMA22f: 频域均衡 (2 2)SDCMA 算法子函数